
Final Project: Semantic Matching and Reasoning with Language Models

Due: June 9, 2026, Tuesday, before 11:59 pm

Project Overview

This project has two parts. Part A studies semantic sentence-pair classification on PAWS, where high lexical overlap can hide important differences in meaning. Part B studies mathematical reasoning on GSM8K, where students adapt and evaluate a small instruction-tuned language model using prompting and LoRA fine-tuning.

- **Part A:** PAWS paraphrase classification with bi-encoders, cross-encoders, and prompt-based inference. **(60 points)**
- **Part B:** GSM8K reasoning with Qwen2.5-1.5B-Instruct, LoRA fine-tuning, and few-shot prompting. **(110 points + up to 10 bonus points)**

Total: 170 regular points, plus up to 10 bonus points.

Part A: PAWS Sentence-Pair Classification

Why PAWS?

Paraphrase detection asks whether two sentences express the same meaning. This is harder than ordinary topic classification because the label can depend on word order, syntactic structure, and how the two sentences align with each other.

We use PAWS (Paraphrase Adversaries from Word Scrambling) [1], a dataset designed to expose this difficulty. Many PAWS sentence pairs share most of their words, but some are true paraphrases and others are not. A model that mostly counts shared tokens can therefore look confident for the wrong reason. The goal of this part is to move from simple dataset diagnostics to contextual models that can compare the two sentences more directly.

There are two main architectures in this part:

- **Bi-encoder:** encode each sentence independently, combine the two sentence vectors, and train a lightweight classifier.
- **Cross-encoder:** concatenate the two sentences and fine-tune one Transformer classifier so attention can operate across both sentences.

The final question asks you to compare these trained encoders against prompt-based classification with a small instruction-tuned language model.

Compute note:

- You should be able to finish the bi-encoder experiments with the free version of Google Colab.
- For cross-encoder fine-tuning and local instruction-model inference, use a GPU runtime.
- Use the Hugging Face `datasets`, `transformers`, and `sentence-transformers` libraries where appropriate.

QUESTION A.1: PAWS Data Inspection and Lexical-Overlap Diagnostic

(20 points: 5 + 5 + 5 + 5)

Load the dataset using:

```
from datasets import load_dataset, ds = load_dataset("paws","labeled_final")
```

Each example contains `sentence1`, `sentence2`, and a binary label `label` (1 = paraphrase, 0 = not paraphrase).

- Dataset inspection.** Report the number of examples in train, validation, and test. Also report the label distribution in each split.
- Qualitative examples.** Find three high-overlap examples with label 0 and three high-overlap examples with label 1. Include the sentence pairs, labels, and a short explanation of what makes the examples difficult.
- Lexical overlap analysis.** For each pair in the test split, compute a simple lexical overlap score, such as Jaccard similarity between the sets of lowercased word tokens in `sentence1` and `sentence2`. Plot the overlap score histogram separately for label 0 and label 1 (two curves/histograms). Briefly interpret the plot. Why is lexical overlap alone not enough to solve PAWS?

- (d) **Modeling hypothesis.** Before training models, write a short hypothesis: which architecture do you expect to work better, a bi-encoder or a cross-encoder? Justify your answer in terms of sentence interaction, runtime, and the structure of the PAWS task.

Bi-encoders and Cross-encoders Transformer-based language models represent text using self-attention, a mechanism that allows each token to attend to other tokens in its input sequence. For sentence-pair classification, the central design choice is whether the two sentences interact inside the Transformer or only after separate sentence embeddings have already been computed.

In a **bi-encoder**, each sentence is encoded independently. This is efficient because sentence embeddings can be cached and reused, but the model cannot directly attend from a word in sentence 1 to a related word in sentence 2. In a **cross-encoder**, the sentence pair is passed jointly to one Transformer classifier. This is slower, but it lets the model compare words and phrases across the pair at every layer.

The encoders in this question are task-specific Transformer models, not general instruction-following LLMs. You will compare the two architectures empirically and analyze the accuracy–efficiency trade-off.

QUESTION A.2: Contextual Encoder Baselines on PAWS

(25 points: 8 + 8 + 4 + 5)

Note: If you are using Google Colab, make sure you change the runtime to GPU (Tesla T4 is sufficient)!

- (a) **Bi-encoder baseline (no fine-tuning).** Choose a small pretrained sentence embedding model (e.g., sentence-transformers/all-MiniLM-L6-v2). Compute embeddings $u = \text{enc}(\text{sentence1})$ and $v = \text{enc}(\text{sentence2})$. Construct pair features however you find sensible. A common recipe is shown below as a *suggestion only* — feel free to try something else (e.g., just $[u, v]$, only $|u - v|$, adding cosine similarity as an extra dimension, etc.). Please explicitly write down your choice in the report and briefly justify it.

$$[u, v, |u - v|, u \odot v]$$

Train Logistic Regression (or Linear SVM) on the **train** split, tune C on **validation**, and report test accuracy, F1 score, and confusion matrix.

- (b) **Cross-encoder fine-tuning.** Fine-tune a small pretrained Transformer encoder (e.g. distilbert-base-uncased) for sequence-pair classification using `AutoModelForSequenceClassification`. To keep compute manageable, train on full training set for only 1–2 epochs. Use the **validation** split for model selection (e.g., learning rate in $\{5 \times 10^{-5}, 2 \times 10^{-5}\}$). Report test accuracy, F1 score, and confusion matrix.
- (c) **Disagreement analysis.** Inspect at least five test examples where the bi-encoder and cross-encoder make different predictions. Which model is correct more often on these examples? What patterns do you observe?
- (d) **Compare encoder models.** Create a table comparing the bi-encoder and cross-encoder on:
- overall test accuracy and F1 score,
 - confusion matrix summary,
 - approximate inference time per 1000 pairs (rough estimate is fine).

Explain the accuracy–efficiency trade-off. Which model would you deploy if accuracy is most important? Which model would you deploy if you need fast retrieval or repeated comparisons?

Prompt-based Inference with Instruction Models Instruction-tuned language models can often follow a natural-language classification prompt without task-specific fine-tuning. This makes them attractive as quick baselines, but they are not always calibrated or stable as classifiers. Their outputs can depend on prompt wording, label names, and decoding settings. In this question, you will compare prompting against the trained encoders from Question A.2 on the same PAWS examples.

QUESTION A.3: Prompt-based Paraphrase Classification with a Small Instruction Model

(15 points: 3 + 4 + 3 + 3 + 2)

Note: If you are using Google Colab, make sure you change the runtime to GPU (Tesla T4 is sufficient)!

This question explores whether **prompting** an instruction-tuned model can solve PAWS without task-specific fine-tuning. You must use an open-source model runnable (no API calls), e.g., Qwen/Qwen3-4B-Instruct-2507 or meta-llama/Llama-3.2-3B-Instruct.

- (a) **Subset selection.** Create a balanced subset of **200** examples from the PAWS **test** split (100 paraphrase, 100 not paraphrase). Keep this subset fixed for all parts of this question.
- (b) **Zero-shot prompt.** Write a prompt that asks the model to decide whether the two sentences are paraphrases. Require the model to output **only one token/word**: `paraphrase` or `not_paraphrase`. You might need to parse the output if the model gives additional text. Report accuracy and a confusion matrix on the 200-example subset. (If the model outputs extra tokens, parse the first valid label token.)
- (c) **Few-shot prompt.** Add 4–6 labeled examples taken **only from the training split** (do not use validation/test labels), and repeat the evaluation. Report accuracy and a confusion matrix.
- (d) **Compare prompting vs fine-tuning.** Compare the best prompting result to your best trained model from Question A.2 (bi-encoder or cross-encoder) on the same 200 examples. Discuss trade-offs in accuracy, runtime, and reproducibility.
- (e) **Prompt sensitivity.** Try one reasonable alternative wording of your zero-shot prompt while keeping the output labels fixed. Does the result change? Briefly discuss what this suggests about using instruction models as classifiers.

In your report: Include your prompts, along with the model name and decoding parameters.

Part B: Teaching a Small Model to Reason with GSM8K

Introduction

State-of-the-art language models capable of complex reasoning (e.g., GPT-5, Claude, DeepSeek-R1) contain hundreds of billions of parameters, require expensive GPU clusters to serve, and are often accessible only through paid APIs. Yet many practical settings (on-device inference, cost-sensitive deployment at scale, classroom use, and privacy-constrained environments) demand models that are *small*, *fast*, and *cheap* to run. The catch is that small models, while efficient, are typically much worse at multi-step reasoning: they may understand the problem statement but fail at planning, intermediate computation, or faithful execution.

This part asks a concrete question: *Can we teach a small model to reason more like a large one, without training from scratch?* Training a new model end-to-end is unrealistic in an academic setting because it requires massive compute (thousands of GPU-hours) and enormous pre-training corpora (trillions of tokens). Instead, we start with a strong **pre-trained** or **instruction-tuned** model that already understands language and then **adapt** it to a target behavior using **Supervised Fine-Tuning (SFT)**. In SFT, the model is shown examples of the desired behavior (here: clear, step-by-step math solutions) and is trained to reproduce that style reliably.

A natural concern is cost. Even if we avoid pre-training, fine-tuning a 1.5B-parameter model can still be expensive if we update *all* weights. Full fine-tuning also risks *catastrophic forgetting*, where the model loses general capabilities while specializing on a narrow task. This motivates **parameter-efficient** methods that (i) train only a tiny fraction of parameters and (ii) leave the original model weights untouched, while still allowing the model to learn a new skill.

Parameter-Efficient Fine-Tuning: Adapting Without Rewriting the Model

Parameter-efficient fine-tuning (PEFT) methods modify how we represent an “update” to the model. Instead of changing the full weight matrices in each transformer layer, we introduce a small set of additional trainable parameters that *steer* the frozen model. This has three practical benefits:

- **Compute efficiency:** fewer trainable parameters means faster training and lower GPU memory use.
- **Storage and sharing:** only small adapter files need to be saved and distributed.
- **Stability:** freezing the base model reduces catastrophic forgetting and preserves general language competence.

In this part, we use one of the most widely adopted PEFT techniques: **Low-Rank Adaptation (LoRA)** [3].

Low-Rank Adaptation (LoRA)

LoRA is based on an empirical observation: the weight updates that matter for fine-tuning are often approximately *low-rank*. Rather than learning a full update $\Delta W \in \mathbb{R}^{d \times d}$ to a weight matrix W , LoRA represents the update as a product of two thin matrices:

$$h = (W + \Delta W)x = Wx + BAx, \quad B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times d}, r \ll d.$$

Here, W is the original (frozen) weight matrix, and only the LoRA parameters A and B are trained. This changes the number of trainable parameters per adapted layer from d^2 to $2dr$, which can be

orders of magnitude smaller when $r \ll d$. Concretely, for a model like Qwen2.5-1.5B with rank $r = 8$ applied to the attention layers (`q_proj`, `k_proj`, `v_proj`, `o_proj`), this corresponds to training roughly **0.14% of the total parameters**—about a $\sim 710\times$ reduction in trainable parameters compared to full fine-tuning.

Practically, LoRA gives us a clean experimental knob: we can increase adapter capacity by increasing the rank r , or trade off compute versus performance by selecting which modules to adapt. Throughout this part, we use the Hugging Face **PEFT** library [5], which provides a simple interface for attaching LoRA adapters to transformer models and training only those adapter parameters.

Why Math Reasoning?

Mathematical reasoning is an ideal testbed for studying model behavior because:

- **Answers are unambiguous.** Unlike open-ended generation, a math answer is either correct or incorrect, so evaluation is straightforward.
- **Multi-step reasoning is required.** Even grade-school word problems demand decomposition, intermediate computations, and a final synthesis step.
- **Failures are diagnosable.** When a model gets the wrong answer, we can often localize the error (e.g., arithmetic slip, flawed logic, or misunderstanding the question).

The GSM8K Dataset

We use **GSM8K** (Grade School Math 8K) [2], a widely-used benchmark of grade-school level math word problems. The dataset contains **7,473 training examples** and **1,319 test examples**. Each problem requires 2–8 steps of elementary arithmetic and reasoning, and the answer is always an integer. Here is a representative example:

GSM8K Example

Question: Betty is saving money for a new wallet which costs \$100. Betty has only half of the money she needs. Her parents decided to give her \$15 for that purpose, and her grandparents twice as much as her parents. How much more money does Betty need to buy the wallet?

Answer: In the beginning, Betty has only $100/2 = \$50$. Betty's grandparents gave her $15 \times 2 = \$30$. This means Betty needs $100 - 50 - 30 - 15 = \$5$ more. 5

Our Model: Qwen2.5-1.5B-Instruct

Throughout this part, we use **Qwen2.5-1.5B-Instruct** [4], a small instruction-tuned language model from Alibaba with 1.5 billion parameters. It has been pre-trained on a large text corpus and instruction-tuned for general chat—but it has *not* been specifically trained for mathematical reasoning. This makes it an ideal “student” model: competent at language but weak at math.

The Gap We Want to Close

To see the problem concretely, consider the following question from the GSM8K test set, and how our base model handles it compared to a strong model (GPT-4.1-mini):

Base Model (Qwen2.5-1.5B-Instruct) — **INCORRECT**

Question: Every day, Wendi feeds each of her chickens three cups of mixed chicken feed. She gives the chickens their feed in three separate meals. In the morning, she gives her flock of chickens 15 cups of feed. In the afternoon, she gives her chickens another 25 cups of feed. How many cups of feed does she need to give her chickens in the final meal of the day if the size of Wendi's flock is 20 chickens?

Model's reasoning (excerpt):

Total feed = Morning feed + Afternoon feed

$15 + 25 = 7$ cups ← Elementary arithmetic error!

...

Final meal = $60 - 7 = 53$

Model's answer: 53 (Ground truth: 20)

The model sets up the problem correctly (each chicken gets 3 cups, 20 chickens $\rightarrow$ 60 cups total) but then makes a catastrophic arithmetic error: $15 + 25 = 7$. This cascades through the rest of the solution.

Now consider GPT-4.1-mini solving a similar problem:

GPT-4.1-mini —

Question: Betty is saving money for a new wallet which costs \$100. Betty has only half of the money she needs. Her parents decided to give her \$15 for that purpose, and her grandparents twice as much as her parents. How much more money does Betty need to buy the wallet?

GPT's reasoning:

1. The wallet costs 100.
2. Betty has half: $\frac{100}{2} = 50$
3. Parents give her 15.
4. Grandparents give $2 \times 15 = 30$
5. Total: $50 + 15 + 30 = 95$
6. Still needs: $100 - 95 = 5$

Answer: 5 ✓

Notice the difference: GPT's solution is *clearly structured*, uses *numbered steps*, performs *correct arithmetic*, and presents the answer in a clean format. **The goal of this part is to teach our small model to reason like this**—through fine-tuning, prompting strategies, and knowledge distillation.

Part Roadmap

- **Task 1:** Benchmark the base model to establish the performance floor.
- **Task 2:** Apply LoRA fine-tuning with GSM8K training data and study scaling behavior.
- **Task 3:** Investigate few-shot prompting and its interaction with fine-tuning.
- **Bonus:** Revisit the base model and test how much prompt wording changes performance.

Setup and Evaluation Protocol

All experiments in this part follow a common evaluation protocol to ensure results are comparable across models and training setups.

System Prompt Guidelines

Because GSM8K problems require multi-step reasoning, we strongly recommend using a **system prompt** during evaluation (and matching it during training). In practice, a well-designed system prompt often improves reliability by enforcing a consistent reasoning style and a predictable final-answer format that can be parsed automatically.

Your system prompt should do two things:

1. **Elicit structured reasoning:** promote step-by-step work and intermediate calculations.
2. **Encourage a consistent answer template:** ask for a clearly marked final answer (using a stable marker) so automated extraction is reliable.

In practice, adding lightweight formatting constraints often improves both (i) reasoning clarity and (ii) evaluation robustness. Common, easy-to-parse conventions include:

- Ending with a clearly labeled final answer such as **Answer:** `<answer>` or **Final:** `<answer>`.
- Using a single LaTeX marker like `\boxed{\dots}` for the final answer.

We encourage you to experiment with 2–3 prompt variants early (e.g., different answer markers or step formatting), then **choose one prompt format and keep it fixed** for the remainder of the project. Keeping the prompt fixed ensures results remain directly comparable across models and training regimes.

Evaluation Protocol

- **Test set:** 100 questions from the GSM8K test split (fixed across all experiments for compute efficiency).
- **Metric:** $\text{Accuracy} = (\text{number of correct answers}) / (\text{total questions})$.

Compute Environment

Note: It is highly recommended you apply for Colab Pro with your student IDs right away, as this part requires heavy GPU usage!

All experiments can be run on Google Colab. Approximate runtimes on a T4 GPU:

- Evaluation (100 questions): ~ 20 minutes
- LoRA training (1,000 examples, 1 epoch): ~ 40 – 50 minutes
- LoRA training (2,000 examples, 1 epoch): ~ 60 – 70 minutes
- Optional larger LoRA training runs: runtime depends on how many extra examples you choose to use.

Task 1 — Baseline: How Good is the Base Model?

Before any fine-tuning, we first measure how well the base Qwen2.5-1.5B-Instruct model solves GSM8K problems *out of the box*. This establishes a **performance floor** that all subsequent interventions (fine-tuning, prompting, distillation) will be compared against.

Task 1.1 Baseline Evaluation

Evaluate the base model on a fixed subset of 100 GSM8K test questions using the project’s standardized prompting and answer-extraction setup.

Note: To reduce inference time, use batching. A batch size of 16 is recommended (and works well on Colab). With batching, 100 questions typically take about 25 minutes. As always, test performance and confirm your batching syntax on a small sample before running the full code.

QUESTION B.1: (7 points) Run the base Qwen2.5-1.5B-Instruct model on 100 GSM8K test questions and report the accuracy. You should expect approximately 35–40%. (*Exact values may vary slightly depending on the prompt format and extraction rule.*)

QUESTION B.2: (8 points) Inspect at least 3 cases where the base model produces an **incorrect** answer. For each example, include:

- The question,
- A short excerpt of the model’s solution highlighting the failure,
- The extracted answer vs. the ground-truth answer.

Classify each failure mode (e.g., arithmetic slip, reasoning/logical error, misunderstanding the problem, formatting/extraction issue). Do you observe any recurring patterns?

Task 2 — LoRA Fine-Tuning on GSM8K

The base model struggles with math. Can we improve it by showing it thousands of worked examples? In this task, you will apply LoRA SFT using the GSM8K training split, starting small and scaling up to study how data quantity affects performance.

Task 2.1 Understanding the Training Pipeline

Before training anything, take time to plan your implementation and understand *what is actually being optimized*. LoRA SFT is conceptually simple, but many failures in practice come from not understanding the data formatting, where adapters are applied, and which tokens contribute to the loss.

Conceptually, the training pipeline has three key ingredients:

1. **Data formatting:** Each GSM8K training example is converted into a chat-style interaction (system prompt + user question + assistant solution). The assistant solution should follow a consistent answer convention so evaluation can extract answers reliably.

2. **LoRA configuration:** LoRA adapters are inserted into selected linear layers (commonly attention projections and MLP projections). The base weights remain frozen, and only the adapter parameters are trained.
3. **Completion-only loss:** The loss is computed only on the assistant’s response tokens (not on the system or user tokens). This focuses training on producing better solutions rather than memorizing the prompt.

Default hyperparameters are summarized below. Use these unless a question explicitly asks you to experiment.

Hyperparameter	Default	Notes
LoRA rank (r)	8	Controls adapter capacity
LoRA alpha	16	Scaling factor (rule of thumb: $2r$)
LoRA dropout	0.05	
Learning rate	2×10^{-4}	Cosine schedule with 5% warmup
Epochs	1	
Per-device batch size	8	
Gradient accumulation	4	Effective batch size = 32
Max sequence length	1024	
Target modules	Attention only	q,k,v,o_proj

QUESTION B.3: (10 points) Pick the three hyperparameters (LoRA rank, LoRA alpha, Gradient accumulation) from the table above. For each one, explain:

- what each hyperparameter controls,
- what you expect to happen if you **increase** it,
- what you expect to happen if you **decrease** it.

Your answer should reflect practical tradeoffs (e.g., compute/memory, stability, overfitting vs. underfitting).

QUESTION B.4: (10 points) Report:

- (a) the total number of parameters in the base model,
- (b) the number of trainable LoRA parameters under the default configuration,
- (c) the percentage of parameters being trained.

Briefly explain why this percentage is small and how LoRA achieves this reduction.

Task 2.2 Training with Increasing Data

The GSM8K training split contains 7,473 question-answer pairs. Rather than starting at full scale, begin with a smaller run to build intuition, validate your setup, and measure the first performance jump.

QUESTION B.5: (15 points) Train a LoRA SFT model using **1,000** training examples. Evaluate on 100 GSM8K test questions and report the accuracy. Include a brief comment on whether the improvement over the baseline matches your expectations.

Scaling up training data improves performance, but it also increases runtime and cost. Before running larger experiments, it is worth forming a hypothesis about whether scaling will meaningfully help.

QUESTION B.6: (10 points) Hypothesis question (write before running larger training): Do you think scaling from 1,000 examples to **2,000** examples is worth the additional compute? What do you expect the accuracy gain to look like (roughly), and why? If you have extra compute, you may optionally try a larger run, but this is not required.

QUESTION B.7: (15 points: 5 + 5 + 5) Now scale up your training data to **2,000** examples. Evaluate the trained model on the same 100-question test subset and report the accuracy. If you choose to run a larger training set, report it as an optional extra result.

Finally, plot accuracy as a function of the number of training examples (x-axis: 0, 1000, 2000; y-axis: accuracy). Describe the trend you observe and comment on diminishing returns in data scaling for SFT.

Task 2.3 Qualitative Analysis

QUESTION B.8: (7 points) Compare the base model and your best SFT model on the same 3 failure examples you identified in Task 1. For each example, show both models' responses side by side. Does the SFT model fix any of these errors?

QUESTION B.9: (3 points) Identify 2 examples where your best SFT model **still fails**. What types of errors persist after fine-tuning? Does the model struggle more with arithmetic, multi-step reasoning, or problem comprehension?

Note: You should expect to reach around 45-50% accuracy by this time.

Task 3 — Few-Shot Prompting

Fine-tuning changes the model's weights. An orthogonal approach is to change the *prompt*: by including a small number of worked examples ("shots") before the actual question, we can steer the model toward a desired reasoning style and a more consistent output format at inference time, without updating any parameters.

Task 3.1 Few-Shot Setup

In k -shot prompting, we prepend k complete (question, solution) pairs as user/assistant turns in the chat before the actual test question. Conceptually, the model sees:

```
[system]    System prompt
[user]      Example question 1
[assistant] Example solution 1
[user]      Example question 2
[assistant] Example solution 2
...
[user]      Actual test question
[assistant] <model generates here>
```

For fairness, use a **fixed pool of k demonstration examples** for all models you evaluate (base, SFT). The goal is to keep the prompt content constant so that differences in performance reflect differences in the model rather than differences in the examples.

Practical note: The quality of your demonstrations matters. A natural question to ask is: *what is the best source for high-quality step-by-step reasoning answers to use as demonstrations?* Yes, that is correct — it *is* the place to go now. Any guesses?

Task 3.2 Few-Shot Experiments

QUESTION B.10: (15 points) Evaluate k -shot prompting (use $k = 3$) on:

1. the base model,
2. your LoRA SFT model trained on 2k examples,

Report the k -shot results alongside the corresponding no-demonstration baseline results, and compute the improvement (Δ).

QUESTION B.11: (10 points) Analyze the effect of few-shot prompting on each model:

- Does few-shot help the **base model**? If not, why might it perform *worse* with demonstrations?
- Does few-shot help the **SFT models**? By how much?
- Which model benefits the most from few-shot prompting, and why?

Bonus — Prompt Playground

(Up to 10 bonus points)

This bonus question revisits Task 1 and asks a simple question: how much can the same base model change when only the prompt changes? This is meant to be a small, fun prompt-engineering challenge.

QUESTION B.12: Prompt Playground Bonus

Use the base Qwen2.5-1.5B-Instruct model, without LoRA fine-tuning, on a fixed subset of **50** GSM8K test questions. Try **2–3 different prompt styles**. You can be creative here: change the model's role, ask it to check its work, force a particular solution style, make it write equations first, or try any other prompt idea that you think might help. For example, you may compare:

- a short direct prompt,
- a step-by-step reasoning prompt,
- a stricter prompt that forces a final answer format such as `\boxed{ANSWER}`.

Keep the model, dataset subset, decoding settings, and answer-extraction rule fixed across prompt variants. Report the accuracy for each prompt in a small table, and briefly analyze the variation you observe. Why do you think one prompt worked better or worse than another? Did the prompt mostly affect reasoning quality, output format, or answer extraction?

Class challenge: We will compare the best prompt accuracies across the class. The highest-scoring prompt(s) will receive full bonus credit; other submissions will receive bonus credit based on both performance and the quality of the analysis.

Optional Exploration — Beyond Scaling

The regular graded portion of Part B ends after Task 3, with the prompt playground above available for bonus credit. The following section is optional and ungraded.

By now, you may have improved over the base model using LoRA SFT and few-shot prompting. You may also have noticed a recurring pattern: **scaling** (more training examples, longer prompts, more shots) helps, but the gains can slow down. At some point, the bottleneck is less about *how much* data you have and more about *how good* the supervision is.

This optional exploration is based on a central idea in reasoning systems: **improving reasoning performance is often a data-quality problem**. If your training solutions are noisy, terse, poorly structured, or inconsistent, then adding more of them may not teach the model the behavior you want. In contrast, training on fewer but higher-quality, well-structured solutions can lead to surprisingly large improvements.

If you are interested, you may try one or more of the following directions:

- **Demonstration quality:** curate a small set of high-quality few-shot demonstrations (clear steps, correct arithmetic, consistent final-answer marker) and measure gains over your current few-shot setup.
- **Prompt and format engineering:** redesign the system prompt and/or answer marker to reduce extraction failures and encourage disciplined reasoning; test a small set of prompt variants and lock one in.
- **Targeted training data selection:** train on a subset of GSM8K that emphasizes harder multi-step problems or known failure modes of the base model.
- **Self-consistency at inference:** sample multiple solutions (temperature > 0) and take a majority vote over extracted answers.
- **Light hyperparameter tuning:** adjust LoRA rank, learning rate, epochs, or target modules; justify expected effects and report what changed.
- **Quality-focused supervision:** generate or obtain a set of higher-quality step-by-step solutions and fine-tune on them (filtering for correctness when possible).

Submission

Please submit a PDF report to Gradescope via BruinLearn. In addition, submit a zip file containing your **report**, your **code**, and a **README** explaining how to run your code to BruinLearn.

The zip file should be named `M146Project_UID1_UID2_..._UIDn.zip`, where `UIDx` values are the student ID numbers of the team members. Only one submission per team is required.

References

- [1] Zhang, Y., Baldridge, J., & He, L. (2019). PAWS: Paraphrase Adversaries from Word Scrambling. *Proceedings of NAACL-HLT 2019*.
- [2] Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., & Schulman, J. (2021). Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*.
- [3] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-Rank Adaptation of Large Language Models. *ICLR 2022*.
- [4] Yang, A., Yang, B., Hui, B., Zheng, B., Yu, B., Zhou, C., ... & Lin, J. (2024). Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115*.
- [5] Hugging Face. PEFT: Parameter-Efficient Fine-Tuning. <https://huggingface.co/docs/peft>.